

Entrega 1

Grupo: Miguel Prieto y Gonzalo Lumeras

Nombre del lenguaje: milu

Sentencias y bloques

Todas terminan con `;`. Las sentencias pueden ser declaraciones, asignaciones, llamadas a funciones... Los bloques de código están delimitados por `{` y `}`, y no necesitan finalizar con `;`, solo las sentencias que lleven dentro. Las indentaciones no son relevantes (los espacios si, para delimitar las palabras), el programa sabe cuál es la estructura mediante `;` y `{}`. Los comentarios son siempre de línea y se denotan con `//` seguido del texto del comentario.

```
sum = 5 + 7;
suma_matrices(matriz1, matriz2, n, m);
if (flag) { sum = 0; return false; } // todo esto se especifica más adelante
```

Identificadores y declaración de variables

Los identificadores están compuestos por letras, números y el carácter `_` sin espacios. Estos pueden ser nombres de variables o de funciones y son únicos en el entorno local (dentro de una misma función o de un mismo bloque). También respetan el ámbito de la variable, y dejan de estar definidos al salir de un bloque, y en caso de dos variables con el mismo identificador se prioriza la que esté definida en el ámbito menos general. Para declarar variables usamos la palabra reservada `var` y especificamos el tipo de la variable después de su identificador, usando `:` para separarlos de esta manera: `var <identificador>:<tipo>`.

```
var i : int;
var flag : bool = true;
var matriz1 : array[n] de int;
```

Funciones

Las funciones se determinan por su identificador y sus parámetros. Se pueden sobrecargar funciones, pero los parámetros de la función deben ser diferentes y al llamarla se usará la función más específica. Las funciones deben tener tipo de retorno, aunque este puede ser `void`, y se especificará detrás de la función, tras el carácter `:`, como en las declaraciones de variables. Las funciones tendrán sus parámetros entre paréntesis detrás del identificador. Irán precedidas por la palabra `def` y antes del bloque que especifique la función irá la palabra `as`, de la siguiente forma: `def <identificador>(<param1> : <tipo1>, <param2> : <tipo2>, ...) : <tipo de retorno> as {<bloque>}`

Los parámetros por defecto serán por valor. Para que sean por referencia se añade el carácter `&` antes del nombre de la variable: `def <identificador>(& <param> : <tipo>)...`
Para llamar una función simplemente se escribe su identificador seguido de los parámetros que queremos poner entre paréntesis, y como siempre se acaba con `;`.

Para definir el valor de retorno de una función se usa *return <expresión>;*, donde *expresión* puede ser una llamada a una función. En el caso de que la función devuelva *void*, no es necesario usar *return*.

```
def mcd(a : int, b : int) : int as { <bloque> }  
def main() : void as { <bloque> }
```

Tipos

Para los tipos declararemos explícitamente el tipo de las variables y usaremos *int*, *bool* y *float* para los tipos básicos enteros, booleanos y reales respectivamente. Una variable de tipo *bool* puede tomar los valores *true* o *false*. Para asignar un valor a una variable se usará *<identificador> = <sentencia>;*, y se puede asignar un valor a una variable en su misma declaración: *var <identificador> : <tipo> = <sentencia>;*. Por otro lado tendremos los siguientes operadores con sus respectivas asociatividades:

Prioridades	Asociatividad
!, - (prefijo)	Asociatividad por la derecha
*, /, %	Asociatividad por la izquierda
+, -	Asociatividad por la izquierda
==, !=, <, >, <=, >=	No asociativos
&&	Asociativo por la izquierda
	Asociativo por la derecha

Sin embargo, se puede especificar el orden de la operación usando paréntesis, (y). Estos pueden ir anidados.

Los arrays siempre serán de tamaño fijo y para declararlos usaremos la sentencia *var <identificadorArray>:array[<numElems>] de <tipo>;*. Esto es cierto incluso al definirlos como parámetros de funciones: deberá especificarse su tamaño en la declaración de la función. Cabe destacar que en la declaración del array, *<tipo>* puede ser otro array. Para acceder a un elemento de un array se usa *<identificador>[i]*.

También hay punteros, que se declaran con el carácter *** antes del tipo. Para cualquier variable, se puede acceder a su dirección de memoria con el carácter *&* seguido de su identificador, y se puede acceder al valor contenido en la dirección de un puntero con el carácter *** seguido del identificador.

```
var puntero : * int;  
puntero = & suma;  
print(* puntero);
```

También existen los registros, que se definen mediante *struct <identificador> as { <bloque> };*, donde *bloque* es un conjunto de declaraciones de variables. Para acceder a los componentes de una variable de tipo registro se usa el operador *.* seguido del identificador de la variable del tipo.

```
struct pos : {  
    var x : int; var y : int;  
}  
var position : pos;  
pos.x = 0; pos.y = 5;
```

```

    print(pos.x);
// Ejemplo
    var puntos : int;
    var partida_ganada : bool;
    var record:array[3] de int;

    def main() : void as {
        puntos = 10;
        partida_ganada = false;
        puntos = -puntos + 5 * 6;
        record[0] = puntos;
        if (puntos > 15 && partida_ganada == false) {
            print(puntos);
        }
    }
}

```

Condicionales y *while*

Para escribir una expresión condicional se sigue la siguiente estructura, donde el *else* es opcional: *if* (<condición>) { <bloque> } *else* { <bloque> }. La condición del *if* es una expresión que tiene como resultado un booleano. Como es natural, si la condición es cierta se ejecuta el código del primer bloque, y en otro caso se ejecuta el código del bloque del *else*.

```
if (flag) { return true; } else { return false; }
```

Para el bucle *while*, también tenemos una condición y el bloque del bucle se ejecuta mientras la condición sea cierta, lo cual se comprueba antes de cada iteración. La sintaxis es: *while* (<condición>) { <bloque> }.

```
while (i < n) { i = i + 1; }
```

También existe el *for*, con la sintaxis habitual, pero no permite la declaración de variables dentro de los argumentos, es decir: *for* ([asignación inicial]; condición; iteración) { <bloque> }

```
for ( i = 0; i < n; i++) { <bloque> }
```

Gestión de errores

Una vez detectado un error se informa al usuario con el siguiente mensaje *Error de tipo <tipo> en <fila>:<columna>*. Cuando se detecta un error se informa al usuario y se salta esa expresión para seguir con la compilación e intentar detectar más errores.

```

// Ejemplo
var cont : int;
cont = true;

```

Error de tipo semántico en 2:12.

```
var cont : int
```

Error de tipo sintáctico en 1:14.

Funciones relevantes

Se definen funciones de entrada y salida para enteros como *print(<argumento>)* y *read(<argumento>)*. Estas funciones escriben y leen un entero, respectivamente, desde la consola.

```
var n : int;  
read(n);  
n = n + 1;  
print(n);
```

También se puede usar *new* seguido de un tipo (o un array) para reservar espacio para un puntero. Para liberar espacio se usa la función *free(puntero)*.

```
var puntero : *int = new int;  
free(puntero);
```

// Ejemplo: máximo común divisor

```
def mcd(a:int, b:int) as {  
    var r:int;  
    r = a % b;  
    if (r == 0) {  
        return b;  
    }  
    else {  
        return mcd(b, r);  
    }  
}
```

// Ejemplo: suma de matrices

```
def suma_matrices(& matriz1:array[MAX] de int, & matriz2:array[MAX] de int,  
n:int, m:int):void as {  
    var i:int;  
    var j:int;  
    while (i < n) {  
        while (j < m) {  
            matriz1[i][j] = matriz1[i][j] + matriz2[i][j];  
            j = j + 1;  
        }  
        i = i + 1;  
        j = 0;  
    }  
}
```